CHAPTER 26

Microwave Anisotropy Satellite

This chapter describes the design of the Wilkinson Microwave Anisotropy (WMAP) Satellite ACS. The system discussed in this chapter is not the ACS that was used on the actual spacecraft. It shows the design process for designing a system that meets the requirements. The actual ACS is discussed in this paper [1]. The attitude determination system is also not discussed in this chapter. It is assumed that the attitude determination system produces an accurate quaternion from the ECI frame to the body frame.

26.1. The WMAP mission

The WMAP mission was designed to make fundamental cosmological measurements of the cosmic background radiation. As such, it needs to be able to point in an arbitrary inertial direction. WMAP produces a map of the cosmic microwave background radiation over the celestial sphere through a fast spin about its spin-axis and a slow precession of its spin axis about the sunline.

26.2. ACS overview

For this example, we replace the reaction wheels with momentum wheels.

The Wilkinson Microwave Anisotropy Probe is a three-axis controlled spacecraft. Fig. 26.1 shows the spacecraft.

The spacecraft attitude control system has

1. Two inertial measurement units (IMUs);
2. Two star trackers;
3. One digital Sun sensor;
4. Twelve coarse Sun sensors;
5. Three reaction-wheel assemblies;
6. A hydrazine monopropellant propulsion system.

The star trackers and IMUs provide the inertial reference. The Sun sensors provide the Sun vector. The reaction wheels are used for control and the thrusters for unloading.

26.3. Control modes

The WMAP spacecraft requires three control modes:

- Mission control mode using reaction wheels with thrusters for momentum unloading. This mode would also include the acquisition function.

ADCS - Spacecraft Attitude Determination and Control
https://doi.org/10.1016/B978-0-32-399915-1.00038-3

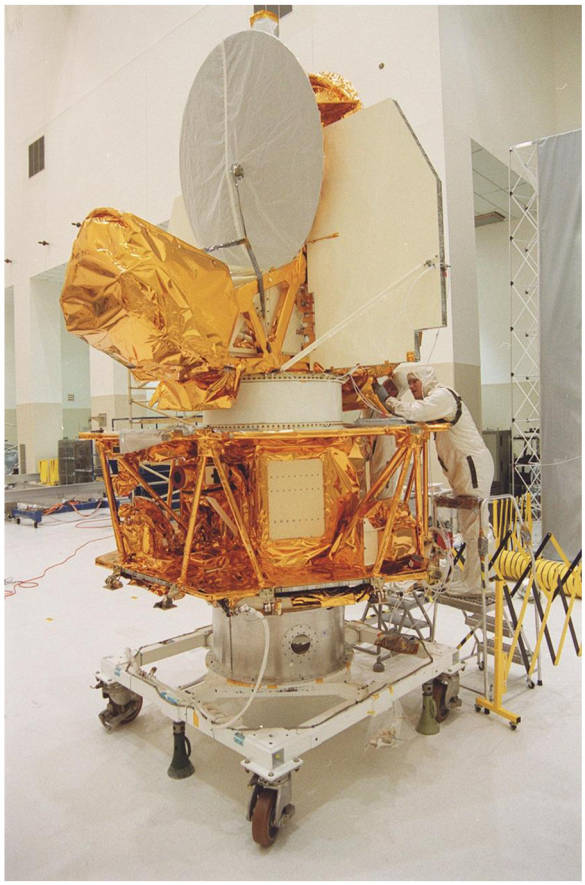


Figure 26.1 The Microwave Anisotropy Probe. Image courtesy of NASA.

- Backup mission control mode with thrusters for three-axis attitude.
- Orbit-adjust mode using thrusters for three-axis attitude control. The reaction wheels are kept in a tachometer mode during orbit changes.

Optionally, a safe-hold mode could be added. The safe mode would be to point the solar panels at the Sun and rotate about the sunline.

26.4. Sensing and actuation

Nominally, WMAP uses gyros for attitude determination and a star tracker to correct for gyro drift and to obtain absolute attitude information. The gyro and star-tracker data could be integrated using an extended iterated Kalman filter. A backup mode using just the star tracker would also be available.

During most of the mission, the reaction wheels are used for attitude control. Secular momentum growth due to solar pressure would be controlled using the thrusters. Momentum unloading could be autonomous or ground commanded. During orbit-adjust modes the thrusters would be used for attitude control since the disturbances due to the orbit-adjust thrusters would quickly saturate the reaction wheels.

26.5. Control-system design

The control system employs tachometer inner loops for the three reaction wheels. The outer loops are two proportional-integral-differential (PID) loops for the transverse axes and a proportional-integral (PI) loop for the spin-axis. The integral term in the PID controllers ensures accurate tracking of the spin-axis target. The inner momentum-wheel loops are PI controllers. A reaction wheel has a current feedback loop to counteract the back-electromotive force. This gives it a linear voltage to torque relationship. A momentum wheel does not have current feedback so as the wheel speed increases, the wheel will produce less torque. Using an inner loop solves this problem. The outer loops output an angular-acceleration demand that is converted to a wheel-speed demand and passed to the reaction-wheel tachometer loops. The inner loops ensure proper reaction-wheel response regardless of bearing friction magnitude, back-electromotive force, and model uncertainty. A sampling rate of 4 Hz was chosen for the digital implementation of this control system.

Fig. 26.2 shows the system. PID loops control roll and pitch that determine the pointing direction of the spin-axis. A PI loop controls the spin rate. The torque demand from the PID and PID are converted to a wheel-speed demand, as explained in the next section. The tachometer loop controls the wheel speed and this provides the torque on the spacecraft.

26.6. Nested loops

It is easiest to understand how the inner loop works with the outer loop if we look at the control of a single axis of the spacecraft.

Assume the dynamical system is composed of two equations for the pitch axis of a momentum bias spacecraft.

$$T = I\ddot{\theta} + J\dot{\Omega} \tag{26.1}$$

$$T_w = J\dot{\Omega} \tag{26.2}$$

where I is the pitch inertia, J is the momentum-wheel inertia, T is an external disturbance torque, θ is the angle that you want to control, Ω is your momentum wheel speed, and T_w is the torque on the wheel produced by an electric motor connecting the

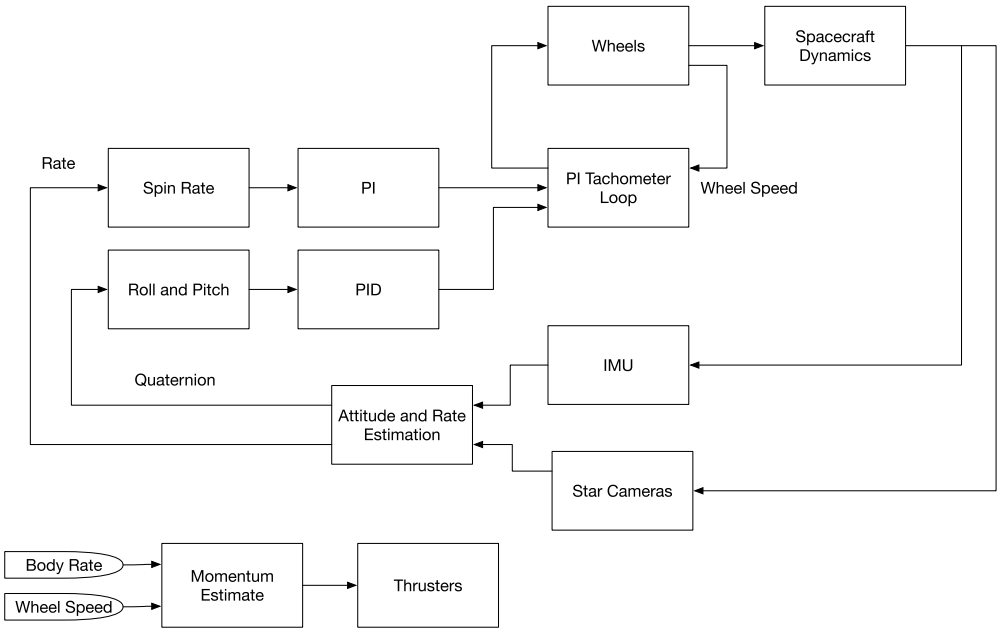


Figure 26.2 Control-system block diagram.

wheel to the spacecraft. You have a measurement of Ω and θ but not T_w. How do we connect the two? Write

$$J\dot{\Omega} = I\left(2\zeta\sigma\dot{\theta} + \sigma^2\theta\right) \tag{26.3}$$

where ζ is the damping ratio and σ is the undamped natural frequency. Our first dynamical equation is now

$$T = I\ddot{\theta} + I\left(2\zeta\sigma\dot{\theta} + \sigma^2\theta\right) \tag{26.4}$$

which is a damped second-order system. Divide by I to clarify the equation

$$\frac{T}{I} = \ddot{\theta} + \zeta\sigma\dot{\theta} + \sigma^2\theta \tag{26.5}$$

Our inner loop commands wheel speed,

$$T_w = Jk\left(\Omega_c - \Omega\right) \tag{26.6}$$

where k is the gain and Ω_c is the commanded wheel speed. The inner-loop dynamical equation is now

$$k\Omega_c = \dot{\Omega} + k\Omega \tag{26.7}$$

which is a damped first-order system. If k is high enough, that is the inner loop is very fast, the wheel speed will equal the commanded speed.

$$\Omega = \Omega_c \tag{26.8}$$

This is the "steady-state" response. How do we determine Ω? Integrate Eq. (26.3)

$$\Omega = \left(\frac{I}{J}\right) \int \left(2\zeta\sigma\dot{\theta} + \sigma^2\theta\right) \tag{26.9}$$

or

$$\Omega_c = \left(\frac{I}{J}\right) \sigma \left(2\zeta\dot{\theta} + \sigma \int \theta\right) \tag{26.10}$$

Our outer loop will control θ. Our inner loop will control Ω.

26.7. Simulation results

Acquisition of an inertial pointing vector was simulated. The mass properties and the actuator characteristics for the baseline MAP spacecraft were not available so generic parameters were used in the simulation. The spacecraft model is for a gyrostat with three reaction wheels. All nonlinear terms are included. The simulation models do not include sensor noise or external disturbances. The spacecraft is initially spinning at 0.464 rpm with the spin-axis aligned with the inertial Z-axis. The desired precession angle command of 22.5 deg is fed into the control loops through a low-pass filter to eliminate transients. The gains of the filter are chosen so that acquisition is completed within ten minutes.

Example 26.1 designs the control system. It creates three controllers

1. Two PIDs for roll and pitch;
2. One PI for yaw rate control;
3. One PI for the momentum-wheel tachometer loops.

The functions in the script use analytic pole placement. The PI and PID have rate filters so that at high frequencies they do not differentiate the incoming signals. This acts as a noise filter.

As can be seen from the plots, the control system successfully acquires and tracks the target. The overall pointing accuracy is better than 0.2 deg. The second plot shows the unit vector for the spin-axis of the spacecraft in three-dimensional space. Initially, it is aligned with the z-axis so its x and y components are zero. After the acquisition, the x- and y-axes trace out a circle in the XY-plane and the z value is constant.

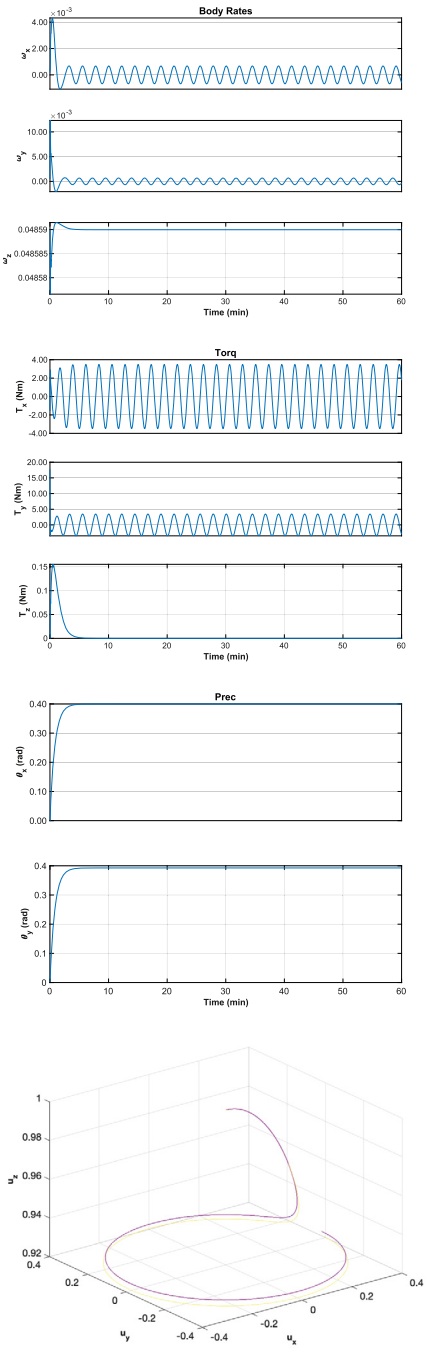
The control system presented here is only part of the overall control architecture that includes not only the other modes but also the attitude determination function, the momentum-unloading system, and the telemetry and command functions. Although

the spacecraft itself is mostly single string, a fault-detection and isolation system might also be included to protect the spacecraft against transient problems.

```
rPMToRPS     = pi/30;
tSamp        = 0.25; % Sampling time
dRHS         = RHSGyrostat;
dRHS.inr     = [2000,0,0;0,2000,0;0,0,4000];
dRHS.inrWheel = 1;

% RWA Tach loops
zeta     = 0.7071;   % Damping ratio
wN       = 1.0;  % Closed loop undamped frequency
[aTL,bTL,cTL,dTL] = PIDesign( zeta, wN, dRHS.
       inrWheel, tSamp, 'Delta' );

% Attitude Loops
zeta     = 0.7071;   % Damping ratio
wN     = 0.5;       % Closed loop undamped
       frequency
wR     = 4.0;       % Rate filter break frequency
tau    =  50;       % Integrator time constant
[aRoll ,bRoll, cRoll, dRoll]  = PIDMIMO( dRHS.inr
       (1,1), zeta, wN, tau, wR, tSamp, 'Delta');
[aPitch,bPitch,cPitch,dPitch] = PIDMIMO( dRHS.inr
       (2,2), zeta, wN, tau, wR, tSamp, 'Delta');

% Rate Loop
zeta         = 1.0;  % Damping ratio
wN           = 0.5;  % Closed loop undamped
       frequency
[aYaw,  bYaw,  cYaw,  dYaw]  = PIDesign( zeta, wN
       , dRHS.inr(3,3), tSamp, 'Delta' );

%% Initialize the control system
xTL              = zeros(3,1);
xRoll            = [0;0];
xPitch           = [0;0];
xYaw             = 0;
spinRate         = 0.464*rPMToRPS;
precRate         = 2*pi/3600;
precAngleDemand  = 22.5;
precAngleGain    = 0.995;
kMM              = 0.00;

% The control distribution matrix converts
% torque demand to angular acceleration demand
aRWA             = eye(3)/dRHS.inrWheel;
```

Example 26.1: Control-loop design code. This sets up the parameters that are used in the simulation.

```
nSim     = 4*3600;
tDist    = [0;0;0];
deg2R = pi/180;

% Initialize controllers
wRWAC          = [0;0;0];
tC             = [0;0;0];
precAngle  = 0;

% [q;omega;omegaRWA]
x  = [[1;0;0;0];[0;0;spinRate];[0;0;0]];
xP = zeros(29,nSim);
t  = 0;
for k = 1:nSim
  qIToB    = x(1:4);
  wCore    = x(5:7);
  wTach    = x(8:10);
  xI       = QTForm(qIToB,[0;0;1]);

  % Momentum Management
  hB       = dRHS.inr*wCore...
           + (dRHS.uWheel.*dRHS.inrWheel')*(wTach+
                 wCore);
  hTotal = QTForm( qIToB, hB );

  % Proportional controller for momentum
  tMM = QForm( qIToB, -kMM*hTotal );

  % Compute the Errors
  precAngle  = precAngleGain*precAngle + (1-
         precAngleGain)*precAngleDemand;
  cP = cos(precAngle*deg2R);
  sP = sin(precAngle*deg2R);
  xT = [sP*cos(precRate*t);sP*sin(precRate*t);cP
       ];
  xTB    = QForm( qIToB, xT );
  rollErr  =  asin(xTB(2));
  pitchErr = -asin(xTB(1));
  yawErr   = wCore(3) - spinRate;

  % The attitude control loops
  xRoll  =  xRoll + aRoll*xRoll + bRoll*rollErr;
  xPitch =  xPitch + aPitch*xPitch + bPitch*
         pitchErr;
  xYaw   =  xYaw + aYaw*xYaw + bYaw*yawErr;
  tC     = -[cRoll*xRoll + dRoll*rollErr;...
            cPitch*xPitch + dPitch*pitchErr;...
            cYaw*xYaw + dYaw*yawErr];

  % Convert torque demand to wheel speed demand
  wDRWA  = -aRWA*tC;

  % Integrate to get wheel speed demand
  wRWAC  = wRWAC + tSamp*wDRWA;

  % The RWA Tach Loops
  wErr = wTach - wRWAC;
  dRHS.torqueWheel = -dTL*wErr - cTL*xTL;
  xTL  = xTL + aTL*xTL + bTL*wErr;
  dRHS.torque = tMM + tDist;

  % Propagate
  x = RK4(@RHSGyrostat,x,tSamp,0,dRHS);
  t = t + tSamp;
  xP(:,k) = [x;tC;hTotal;tMM;...
    acos([xI(3);xT(3)]);...
    rollErr;pitchErr;xI;xT];
end

yL = [ dRHS.states(:)' ...
{'T_x␣(Nm)'} {'T_y␣(Nm)'}  {'T_z␣(Nm)'} ...
{'H_x␣(Nms)'} {'H_y␣(Nms)'} {'H_z␣(Nms)'} ...
{'T_x␣(Nm)'} {'T_y␣(Nm)'}  {'T_z␣(Nm)'}...
{'\theta_x␣(rad)'} {'\theta_y␣(rad)'}...
{'Roll␣(rad)'} {'Pitch␣(deg)'}...
{'u_x'} {'u_y'} {'u_z'} ];

t = (0:nSim-1)*tSamp; k = 1:4;
TimeHistory(t,xP(k,:),yL(k),'Quat');k = 5:7;
TimeHistory(t,xP(k,:),yL(k),'Body␣Rates');k=k+3;
TimeHistory(t,xP(k,:),yL(k),'RWA');k=k+3;
TimeHistory(t,xP(k,:),yL(k),'Torq');k=k+3;
TimeHistory(t,xP(k,:),yL(k),'Momentum');k=k+3;
TimeHistory(t,xP(k,:),yL(k),'MM␣Torq');k=[20 21];
TimeHistory(t,xP(k,:),yL(k),'Prec');k=[22 23];
TimeHistory(t,xP(k,:),yL(k),'Errors');

j1 = 24:26; j2 = 27:29;
PlotV([xP(j1,:);xP(j2,:)],yL{j1},'Spin␣Axis')

Figui
```



Example 26.2: Spin up and acquisition simulation. The spacecraft acquires the target quickly.

References

[1] L. Markley, S. Andrews, J. O'Donnell, D. Ward, A. Ericsson, The Microwave Anisotropy Probe Attitude Control System, 10 2011.